

MA 223

Data Structure and Applications Lab

Team Project 6 - Locality Sensitive Hashing

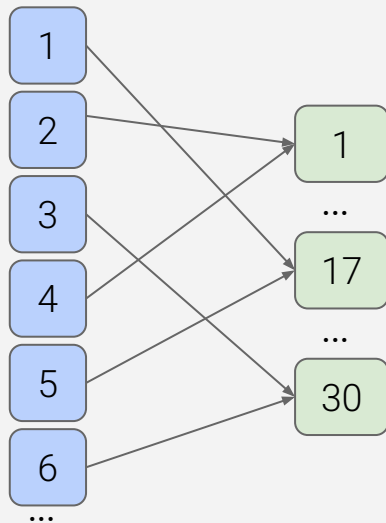
What is Locality Sensitive Hashing?

Idea:

The core idea behind LSH is to hash data points in such a way that similar items are mapped to the same hash buckets with high probability. This “similar items” could refer to pages with similar words, images with similar features.

Interesting to note that, unlike traditional hashing, where we aim to minimize hashing collisions, for LSH we try to maximize collisions (by trying to map them to the same hash bucket) for similar items.

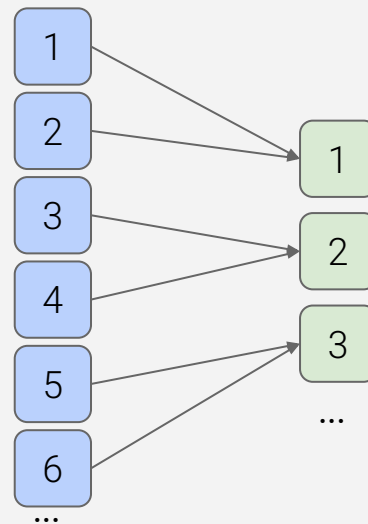
Traditional Hash Function



Applications:

Checksums
Password Hashing
Hash Tables

Locality Sensitive Hash



Applications:

Dimensionality Reduction
Nearest Neighbor Search

Formal Definition

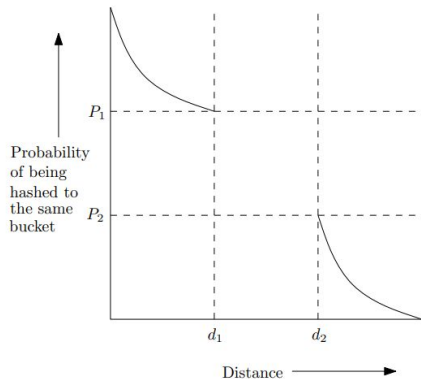
Locality Sensitive Hashing:

We consider an LSH family F defined for a metric space $M = (M, d)$, a threshold $r > 0$ and an approximation factor $c > 1$. The family F of functions $h: M \rightarrow S$ maps elements from the metric space to buckets $s \in S$. For any two points $p, q \in M$, using a function $h \in F$ chosen randomly:

If $d(p, q) \leq r$, then $h(p) = h(q)$ with probability $\geq p_1$

If $d(p, q) \geq cr$, then $h(p) = h(q)$ with probability at most p_2

Crucial Condition: $p_1 > p_2$



The LSH Pipeline



Explanation/Pseudocode

Phase 1: Setup (Choosing the Rules)

FOR each Table (1 to L):

 Pick K random numbers between 0 and D.

 Save these numbers as the RULE for this table.

 // Example RULE for Table 1 with K = 3: "Look at bits 5, 12, and 99"

Explanation/Pseudocode

Phase 2: Indexing (Organizing existing data)

FOR each Point P in the Dataset:

FOR each Table (1 to L):

1. Look at the "Rule" for this table.
2. Extract the bits from Point P at those positions.
3. Concat them together to make a Signature (e.g., "101").
4. Put Point P's ID into the bucket labeled "101".

Explanation/Pseudocode

Phase 3: Querying

FUNCTION Find_Neighbor(Query Q):

 Candidate_Set = Empty List

 // Step A: Gather Suspects

 FOR each Table (1 to L):

1. Calculate "Signature" for Q using this table's Rule.
2. Go to that bucket.
3. Copy all Point IDs found there into Candidate_Set.

 // Step B: Verify (Mini Brute-Force)

 Best_Match = Null

 FOR each Point C in Candidate_Set:

 Distance = Calculate_Hamming_Dist(Q, C)

 IF Distance is the smallest seen so far:

 Update Best_Match

 RETURN Best_Match

Implementation Demonstration

Evaluation Metrics

Example 1

Configuration: N=5000, D=128, K=10, L=5			
Generating Dataset X (5000 binary vectors)			
Building LSH Index			
[LSH] Indexing 5000 points across 5 tables			
Index Build Time: 27 ms			
=====			
PERFORMANCE EVALUATION: Comparing Methods A, B, and C over 100 Random Queries			
=====			
Metric	[A] Brute Force	[B] LSH Union	[C] LSH Single

-			
Avg Time (microsec)	8384.7	79.92	26.3
Avg Candidates Checked	5000	34.55	7.25
Exact Recall (%)	100%	9	2
Approx Ratio (1.0=Best)	1.00	1.11675	1.2157

Example 2

Configuration: N=5000, D=128, K=6, L=10			
Generating Dataset X (5000 binary vectors)			
Building LSH Index			
[LSH] Indexing 5000 points across 10 tables			
Index Build Time: 31 ms			
=====			
PERFORMANCE EVALUATION: Comparing Methods A, B, and C over 100 Random Queries			
=====			
Metric	[A] Brute Force	[B] LSH Union	[C] LSH Single

-			
Avg Time (microsec)	8127.21	1726.65	181.21
Avg Candidates Checked	5000	911.62	99.78
Exact Recall (%)	100%	76	14
Approx Ratio (1.0=Best)	1.00	1.01189	1.0966

Complexity Analysis

1. Brute Force (Deterministic)

Baseline approach

Preprocessing Time: $O(1)$ (None)

Query Time: $O(N \cdot D)$

Because we compute the Hamming distance (requires D operations) for all N points.

Space Complexity: $O(N \cdot D)$

Only the raw dataset is stored.

Complexity Analysis

2. Standard LSH (Union of L Tables)

A balanced approach trading memory for search speed.

Preprocessing Time: $O(N \cdot L \cdot K)$

For each of the N points, we compute L hashes, each using K bits.

Query Time: $O(L \cdot K + C \cdot D)$

$O(L \cdot K)$: computing hash signatures for the query across L tables

$O(C \cdot D)$: computing Hamming distance only for the unique candidate set

Space Complexity: $O(N \cdot D + N \cdot L)$

$O(N \cdot D)$: raw dataset storage

$O(N \cdot L)$: hash tables storing point IDs across L tables

Complexity Analysis

3. Stochastic LSH (Single Random Table)

Fastest runtime but also lowest recall.

Preprocessing Time: $O(N \cdot L \cdot K)$

(The index is still fully constructed.)

Query Time: $O(K + C_{\text{single}} \cdot D)$

Why?

$O(K)$: only one hash signature is computed

$O(C_{\text{single}} \cdot D)$: verify candidates from one bucket only

C_{single} is approximately $\frac{1}{L}$ of the candidate size from the standard LSH method.

Space Complexity: $O(N \cdot D + N \cdot L)$

Because the full index still exists.

Expected number of candidates

Step 1: Probability of Collision in One Table

If the Hamming distance between two signatures is h , the probability of matching one bit is:

$$1 - \frac{h}{D}$$

Therefore, the probability of matching all K bits is:

$$P_{\text{single}} = \left(1 - \frac{h}{D}\right)^K$$

Expected number of candidates

Step 2: Probability of Collision in Any of the L Tables

The probability of failing to collide in one table is:

$$1 - P_{\text{single}}$$

The probability of failing to collide in all L tables is:

$$(1 - P_{\text{single}})^L$$

Therefore, the probability of at least one collision is:

$$P_{\text{collision}} = 1 - \left(1 - \left(1 - \frac{h}{D}\right)^K\right)^L$$

Expected number of candidates

Step 3: Expected Number of Matches

$$E[\text{Candidates}] = N \cdot P_{\text{collision}}$$

Example Calculation (Using Our Parameters)

Parameters: $N = 5000$, $D = 128$, $K = 6$, $L = 10$

Match in One Table ($K = 6$ bits):

$$0.5^6 = 0.015625 \quad (1.5625\%)$$

Match in Any Table ($L = 10$):

$$1 - (1 - 0.015625)^{10}$$

$$\approx 1 - 0.854 = 0.146$$

Expected Number of Candidates:

$$E[\text{Candidates}] = 5000 \times 0.146 = \mathbf{730} \text{ candidates}$$

Extension to R^n Metric Space

With distance metric = Cosine Similarity

Our previous implementation works on **Binary Vectors** using **Hamming Distance**.

- To capture the "angle" or direction of a vector, we slice the high-dimensional space with random cuts.
- These cuts are called **Random Hyperplanes**.
- A hyperplane passing through the origin is defined by its **Normal Vector**.

For each random hyperplane:

1. We project our data point a random hyperplane.
2. **Positive Result** : Point is "Above" the plane maps to Bit **1**.
3. **Negative Result (<0)**: Point is "Below" the plane maps to Bit **0**.

By repeating this K times with different planes, we generate a K -bit signature.

Method Comparison

Difference between Stochastic Methods(like KD-trees) and LSH

Feature	Brute Force	KD-Trees	LSH (Ours)
Strategy	Exhaustive Scan	Space Partitioning	Probabilistic Bucketing
Low Dim	$O(N \cdot D)$	Fastest	Fast but less accurate
High Dim ($D > 100$)	$O(N \cdot D)$	$\sim O(N \cdot D)$	$\ll O(N \cdot D)$ Fastest
Accuracy	100%	100%	$< 100\%$
Primary Flaw	Too slow at scale	Curse of Dimensionality	False Negatives (Misses)

Applications of LSH

- Recommendation Systems
- Computer Vision: Visual Search & Matching
- Near-Duplicate Document Detection (uses SimHash)
- Clustering
- Anomaly Detection

References

- https://www.youtube.com/watch?v=NJaJrVF__B4
- <https://www.youtube.com/watch?v=R-iFka68ZwM>
- <https://cse.iitkgp.ac.in/~animeshm/algoml/lsh.pdf>
- https://medium.com/@sarthakjoshi_9398/understanding-locality-sensitive-hashing-lsh-a-powerful-technique-for-similarity-search-a95b090bdc4a
- <https://scikit-learn.org/0.16/modules/generated/sklearn.neighbors.LSHForest.html>